CSA0315 – DATA STRUCTURES
C04-AT1-COMPARATIVE ANALYSIS

NAME : MONIGA V
REGISTER NO : 192521184

1. Hashing vs. Sorting-based Search (Binary Search)

For a system that stores large volumes of data and performs frequent search operations, the choice between a hash table and a sorted array with binary search has major consequences for performance, memory, and maintainability.

a) Time Complexity

Operation	Hashing	Binary Search (on sorted array)
Search (average case)	O(1)	O(log n)
Search (worst case)	O(n) — with many collisions	O(log n)
Insertion	O(1) average	O(n) — needs shifting to keep order
Deletion	O(1) average	O(n) — needs shifting to keep order
Prerequisite	None (works on unsorted data)	Data must be sorted first, O(n log n)

b) Space Usage

- Hashing: Requires extra memory for the hash table structure, typically sized larger than the number of elements (load factor kept below 1) to minimize collisions. Chaining adds pointer/overhead per bucket.
- Binary Search: Needs only the array itself (no auxiliary structure), so it is more space-efficient when data is already stored contiguously.

c) Update Operations (Insert/Delete)

- Hashing: Insertions and deletions are fast, O(1) on average, since position is computed directly by the hash function.
- Binary Search: Insertions and deletions are costly, O(n), because sort order must be preserved — this typically means shifting elements or rebuilding part of the array.

d) Real-World Applicability

- Hashing is preferred for: caches, symbol tables in compilers, database indexing (hash indexes), duplicate detection, and any system with frequent lookups plus frequent updates.
- Binary Search is preferred for: range queries (e.g., "find all values between X and Y"), ordered traversal, systems where data is mostly static after an initial load (e.g., static lookup tables, read-heavy reference data), and when memory overhead must be minimal.

Conclusion — Which is Better for Dynamic Systems?

For a dynamic system — one with large data and frequent insertions, deletions, and searches — Hashing is generally the better choice. It offers O(1) average time for search, insert, and delete, whereas binary search requires the data to remain sorted, making updates expensive at O(n) per operation. Binary search only becomes preferable when the workload requires ordered access or range queries, or when the dataset is largely static and memory efficiency matters more than update speed.

2. Quick Sort vs. Merge Sort

Both Quick Sort and Merge Sort are efficient, comparison-based, divide-and-conquer sorting algorithms, but they differ significantly in guarantees, memory behavior, and practical speed.

a) Best, Average, and Worst-Case Complexity

Case	Quick Sort	Merge Sort
Best Case	O(n log n)	O(n log n)
Average Case	O(n log n)	O(n log n)
Worst Case	$O(n^2)$ — occurs with poor pivot choices (e.g., already sorted data with a naive pivot)	O(n log n) — guaranteed regardless of input

b) Stability

- Quick Sort: Not stable by default — equal elements can be reordered relative to each other during partitioning.
- Merge Sort: Stable — the merge step preserves the relative order of equal elements, which matters when sorting records by one key while preserving prior ordering on another.

c) Memory Usage

- Quick Sort: In-place, requiring only O(log n) auxiliary space for the recursion stack (with good pivot choices).
- Merge Sort: Not in-place, requiring O(n) additional space for the temporary arrays used during merging.

d) Practical Performance

- Quick Sort: Generally faster in practice due to better cache locality, in-place partitioning, and lower constant factors.
- Merge Sort: Slightly slower in practice due to the overhead of allocating and copying auxiliary arrays, but its performance is far more predictable.

Why Quick Sort Is Often Preferred Despite Its Worst-Case Complexity

- Average-case dominance: Its O(n log n) average case is achieved with smaller constant factors than Merge Sort, making it faster on typical, real-world data.
- In-place sorting: It needs only O(log n) extra space, versus Merge Sort's O(n), which matters for memory-constrained or very large datasets.
- Cache efficiency: Its partitioning works on contiguous memory blocks in place, leading to better CPU cache utilization than Merge Sort's array copying.

- Mitigated worst case: The $O(n^2)$ worst case is rare in practice and can be avoided using randomized pivot selection, median-of-three pivoting, or switching to introsort (which falls back to heap sort when recursion depth gets too large) — techniques used in real standard library implementations.
- No extra allocation overhead: Avoiding auxiliary array allocation reduces both memory pressure and garbage collection overhead in managed languages.

As a result, many standard library sort functions (e.g., C's qsort, and hybrid approaches like Introsort used in C++ STL) are built around Quick Sort's core approach, while Merge Sort is favored instead when stability is required or when working with data structures like linked lists, or in external sorting where $O(n)$ extra space and guaranteed worst-case performance are more valuable than raw average-case speed.